Développement et déploiement d'applications intelligentes

Rattrapage du TP1 au TP5

Démarche complète, commande par commande,
avec les résultats attendus

TP1	Cadrer l'application et préparer l'environnement
TP2	Entraîner, évaluer et sauvegarder le modèle
TP3	Exposer le modèle par une API
TP4	Tests approfondis et qualité
TP5	L'interface Streamlit

A. Larhlimi

Octobre 2026

Table des matières

1 Vue d'ensemble **3**
1.1 Le fil des cinq TP . . . 3
1.2 L'architecture du dépôt . . . 3
1.3 Comment lire ce document . . . 4

2 Les ressources à télécharger **4**
2.1 Décompresser sans piège . . . 4

3 Préparer le poste **5**
3.1 Python 3.11, Git et VS Code . . . 5
3.2 L'identité Git de chaque membre . . . 5
3.3 PowerShell et cmd . . . 6
3.4 Les comptes GitHub . . . 6

4 Qui fait quoi dans le binôme **6**

5 TP1 : cadrer l'application et préparer l'environnement **7**
5.1 Étape 1 : récupérer le kit (membre A, T0) . . . 7
5.2 Étape 2 : l'environnement (membre A, T1) . . . 7
5.3 Étape 3 : les fiches de cadrage (à deux, 50 min, T2) . . . 8
5.4 Étape 4 : le dépôt GitHub (membre A, T3) . . . 8
5.5 Étape 5 : rejoindre le dépôt (membre B, T4) . . . 9
5.6 Étape 6 : un commit par membre (membre A, T5) . . . 9
5.7 Étape 7 : la vérification finale (à deux, T6) . . . 9
5.8 Étape 8 : le test de l'enseignant, sur un clone frais (T7) . . . 10

6 TP2 : entraîner, évaluer et sauvegarder le modèle **10**
6.1 Installer le kit (C1) . . . 10
6.2 Générer, entraîner, évaluer (C2 à C4) . . . 10
6.3 Le vrai jeu de données, les tests, le push (C5) . . . 11

7 TP3 : exposer le modèle par une API **11**
7.1 Installer le kit (T14) . . . 11
7.2 Faire répondre l'API (T15 à T18) . . . 12
7.3 Pousser et vérifier la CI (T19) . . . 12
7.4 L'API sans modèle (T20), puis le compte rendu . . . 12

8 Les tests de l'API **13**
8.1 Dans le navigateur : la page `/docs` . . . 13
8.2 En ligne de commande . . . 13
8.3 Les tests automatiques . . . 14
8.4 Lire un refus 422 . . . 15

9 TP4 : tests approfondis et qualité **15**
9.1 Avant la séance : le jeu d'or . . . 15
9.2 Étape A : installer le kit et mesurer le départ (T21) . . . 15
9.3 Étape B : le contrat d'entrée (T22, T23) . . . 16
9.4 Étape C : l'abstention et le jeu d'or (T24, T25) . . . 16
9.5 Étape D : l'API telle qu'elle tourne (T26) . . . 17
9.6 Étape E : qualité statique, crochet et CI (T27) . . . 17

10 TP5 : l'interface Streamlit **18**
10.1 Le rôle de l'interface . . . 18
10.2 Installer le kit (T28) . . . 19
10.3 La structure de l'interface (étape B, 20 min) . . . 19
10.3.1 Trois fichiers, trois rôles . . . 19
10.3.2 La page, zone par zone . . . 19
10.3.3 Le chemin d'une requête . . . 20
10.3.4 Le code de la page, bloc par bloc . . . 20
10.3.5 Le client : un contrat simple pour la page . . . 22
10.3.6 Le cycle de Streamlit . . . 22
10.3.7 Comment les tests se branchent . . . 23
10.4 Lancer et analyser (T29 à T31) . . . 23
10.5 Provoquer les erreurs (T32) . . . 23
10.6 Un fichier d'avis (T33) . . . 23
10.7 Compte rendu, push, CI (T34) . . . 24

11 Vérification et notation par l'enseignant **24**
11.1 L'accès aux dépôts . . . 24
11.2 En continu : la CI . . . 24
11.3 À la date de remise : la relève automatique . . . 24
11.4 Le clone frais, à la main . . . 25
11.5 Les grilles de notation . . . 25
11.6 Les signaux d'alerte . . . 26

12 Les messages d'erreur rencontrés **26**

A Récapitulatif des contrôles **28**

B PowerShell, cmd et bash **30**

1 Vue d'ensemble

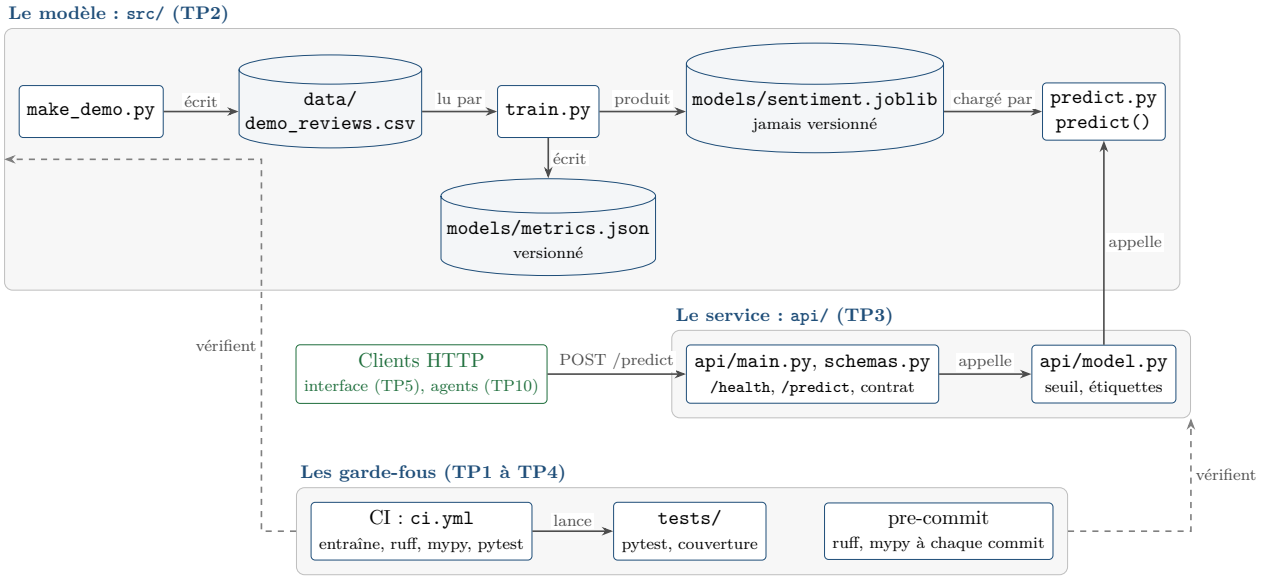
Ce document conduit un binôme du poste vide jusqu'au dépôt `sentiment-app` de fin de TP5 : l'environnement et le cadrage (TP1), un modèle de sentiment (TP2), une API qui l'expose (TP3), les tests et outils qui la protègent (TP4), puis l'interface qui l'appelle (TP5). Chaque étape donne la commande à lancer, puis le résultat exact à obtenir avant de continuer. Les deux dernières parties s'adressent à l'enseignant : comment il vérifie, et comment il note.

1.1 Le fil des cinq TP

TP	Sem.	Ce que le dépôt gagne	Contrôles	Tests à la fin
TP1	1	Environnement Python 3.11, structure du dépôt, trois fiches de cadrage, dépôt GitHub à deux auteurs	T0 à T7	38 passed
TP2	2	Jeu de démonstration, entraînement, évaluation, fonction `predict()`	C1 à C5	42 passed
TP3	3	API FastAPI (`/health`, `/predict`), contrat Pydantic, CI qui entraîne le modèle	T14 à T20	48 passed
TP4	4	Tests aux limites, abstention, tests d'intégration, ruff, mypy, pre-commit, CI étendue	T21 à T27	aucun `failed`
TP5	5	Interface Streamlit : un avis ou un fichier d'avis, verdicts, abstention, erreurs traduites	T28 à T34	aucun `failed`

Avec le corrigé du TP4, on obtient 83 passed et 2 xfailed, puis 95 passed et 2 xfailed après le TP5 ; les nombres d'un binôme peuvent être plus grands, jamais avec un `failed`.

1.2 L'architecture du dépôt



Le dépôt suit un seul chemin : des avis, un modèle entraîné, une fonction de prédiction, une API qui l'expose. Trois règles en découlent, et la notation les vérifie :

1. L'API n'entraîne rien : elle appelle `predict()` du TP2, qui charge `models/sentiment.joblib` une seule fois.

2. Le modèle n'est jamais versionné : il se réentraîne en une commande. Seule sa mesure, `models/metrics.json`, entre dans Git.
3. Les clients (l'interface du TP5, les agents du TP10) passent par l'API, jamais par le fichier `.joblib`.

1.3 Comment lire ce document

— Les commandes sont écrites pour PowerShell, sous Windows. Sauf mention contraire, elles se lancent dans le dossier du dépôt, `C:\IA-DEV\sentiment-app`, avec l'environnement virtuel activé. Les équivalents pour cmd et bash sont en annexe B.

— **Une commande par ligne.** Quand un tableau donne deux commandes dans la même case, lancez-les l'une après l'autre : les mots qui les relient ne font pas partie des commandes.

— Les encadrés verts donnent le résultat attendu, les orange les pièges, les bleus le point de vue de l'enseignant.

— Ne passez à l'étape suivante que lorsque le contrôle est vert. Un échec annoncé comme normal l'est vraiment : il désigne le travail qui reste à faire.

2 Les ressources à télécharger

Ressource	TP	Pour qui	Contenu
Kit du TP1 : dépôt du cours, bouton « Code » puis « Download ZIP » (`sentiment-app-main.zip`)	1	binôme	Structure du dépôt, 38 tests (23 d'environnement, 12 de cadrage, 3 d'historique Git), fiches à remplir, scripts `setup` et `check_setup`, CI
`tp2-semaine2-kit.zip`	2	binôme	`src/` (`make_demo`, `train`, `evaluate`, `predict`), `tests/-test_model.py`, notebook Colab, énoncé, marche à suivre
`kit-tp3.zip`	3	binôme	`api/`, `tests/test_api.py`, `docs/semaine3.md`, énoncé, marche à suivre ; le script d'installation adapte `py-test.ini` et la CI
`tp4-tests-qualite-kit.zip`	4	binôme	Outils (`requirements-dev.txt`, `pyproject.toml`, pre-commit), courbe du seuil, bilan du jeu d'or, tests à compléter, `ci-tp4.yml`, énoncé, marche
`tp5-interface-kit.zip`	5	binôme	`ui/client.py`, `ui/app.py`, tests du client et de la page, fichier d'avis d'exemple, énoncé, marche
`journal-tp1-tp5.md`	1 à 5	binôme	Le journal de bord à compléter, avec ses captures d'écran
`tp4-paquet.zip`	4	enseignant	Le kit, le corrigé complet, les valeurs mesurées, le calendrier de diffusion. **Ne pas distribuer.**
`releve_binomes.py`, `binomes.csv`	tous	enseignant	La relève automatique des dépôts de la classe

Le dépôt du cours est https://github.com/larhlimi2023/sentiment-app. Les archives des TP2 à TP4 sont fournies par l'enseignant.

2.1 Décompresser sans piège

L'Explorateur de Windows ajoute un dossier en décompressant : `kit-tp3.zip` devient `kit-tp3\kit-tp3`, et les chemins relatifs des scripts ne mènent plus au dépôt. Décompressez donc avec

PowerShell, directement dans le dossier de travail, et donnez toujours aux scripts d'installation le chemin complet du dépôt.

```
New-Item -ItemType Directory -Force C:\IA-DEV
Expand-Archive "$HOME\Downloads\tp2-semaine2-kit.zip" -DestinationPath C:\IA-DEV -Force
Get-ChildItem C:\IA-DEV
```

Résultat attendu

Un dossier par kit, directement dans `C:\IA-DEV` : `tp2-semaine2`, `kit-tp3`, `tp4-tests-qualite`, `tp5-interface`, à côté de `sentiment-app`. Jamais à l'intérieur du dépôt.

Archive introuvable

Si `Expand-Archive` répond que le chemin n'existe pas, l'archive n'est pas dans Téléchargements, ou le navigateur l'a renommée (`(1)` ajouté au nom). Cherchez-la, puis copiez le chemin le plus récent entre guillemets :

```
Get-ChildItem $HOME -Recurse -Filter "tp4-tests-qualite-kit*.zip" -ErrorAction
    ↪ SilentlyContinue | Select-Object FullName, LastWriteTime
```

3 Préparer le poste

3.1 Python 3.11, Git et VS Code

```
py -0
py -3.11 --version
git --version
```

Résultat attendu

`py -0` liste une version 3.11, et `py -3.11 --version` affiche `Python 3.11.x`. Sinon : `winget install Python.Python.3.11`, ou l'installateur 3.11.9 de python.org.

Pourquoi 3.11 et pas une version plus récente

Les tests du TP1 vérifient la version de Python. Surtout, pandas 2.2 et scikit-learn 1.5, épinglés par le cours, n'ont pas de version précompilée pour les Python plus récents : pip essaie alors de les compiler et échoue (`Compiler cl cannot compile programs`, `metadata-generation-failed`). C'est toujours `py -3.11 -m venv .venv` qui crée l'environnement.

3.2 L'identité Git de chaque membre

Chaque membre, sur son propre poste :

```
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@ecole.ma"
```

C'est ce nom que `git shortlog` compte, et le test d'historique du TP1 exige deux auteurs distincts. Si une seule personne joue les deux rôles sur un même poste, l'identité se règle pour un seul dépôt, sans `-global`.

3.3 PowerShell et cmd

Une invite qui commence par `PS C:\...` est PowerShell ; `C:\...>` sans `PS` est l'invite de commandes (cmd). Ce guide est écrit pour PowerShell, et plusieurs commandes n'existent pas dans cmd (`Rename-Item`, `Get-Content`, `$env:TEMP`). Dans VS Code, choisissez PowerShell comme terminal.

Une fois par poste, autorisez l'activation des environnements virtuels :

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

3.4 Les comptes GitHub

Chaque membre a son compte. L'enseignant utilise le compte `Larhlimiabd-ai` (affiché « Larhlimi-AI-2026 »). Quand un poste a servi à plusieurs comptes, Windows peut envoyer les mauvais identifiants ; mettez alors le compte dans l'adresse du dépôt (`https://<compte>@github.com/...`), ou effacez les identifiants mémorisés avec `cmdkey /delete:git:https://github.com`.

4 Qui fait quoi dans le binôme

Les deux membres travaillent dans le même dépôt, créé par le membre A. Le membre B y est collaborateur : il n'a ni fork ni dépôt à lui. L'enseignant lit l'historique, et chaque membre doit avoir commité dans chaque TP.

TP	Membre A	Membre B	À deux
TP1	Récupère le kit, crée l'environnement, crée le dépôt GitHub privé, invite B et l'enseignant, premier push avec `requirements.lock`	Accepte l'invitation, clone, crée son environnement, commite sa ligne du `README.md`	Les trois fiches de cadrage ; la vérification finale ; le clone frais
TP2	Installe le kit, génère les données, entraîne, évalue, pousse	Récupère le travail, réentraîne sur son poste, contrôle C5	La lecture des erreurs et `docs/semaine2.md`
TP3	Installe le kit, réentraîne, lance les tests, pousse	Récupère, réentraîne, fait les essais de l'API (T15 à T20)	Les deux questions de `docs/semaine3.md`
TP4	Étapes B et C : contrat d'entrée, courbe du seuil, abstention	Étapes D et E : intégration, contrat typé, ruff, pre-commit, CI	Le jeu d'or (dix avis chacun) et `docs/semaine4.md`
TP5	Installe le kit, lance l'API et la page, essais T29 à T31	Récupère, provoque les erreurs (T32), analyse le fichier d'avis (T33)	`docs/semaine5.md` et la vérification de la CI

Trois habitudes évitent presque tous les conflits : `git pull` avant de commencer, un commit par tâche avec un message explicite, `git push` à la fin. Au TP4, répartissez le travail par fichiers, pour

ne jamais modifier le même fichier en même temps. Le modèle n'étant pas versionné, chaque membre le réentraîne sur son poste après un `git pull`.

5 TP1 : cadrer l'application et préparer l'environnement

Le TP1 produit un dépôt GitHub privé, à deux auteurs, qui contient l'environnement, la structure du projet et trois fiches de cadrage. Huit étapes, huit contrôles (T0 à T7), environ deux heures.

5.1 Étape 1 : récupérer le kit (membre A, T0)

Sur le dépôt du cours, « Code » puis « Download ZIP ». Ne clonez pas ce dépôt : un clone emporterait l'historique de l'enseignant, et le test d'historique compterait ses commits.

```
New-Item -ItemType Directory -Force C:\IA-DEV
Expand-Archive "$HOME\Downloads\sentiment-app-main.zip" -DestinationPath C:\IA-DEV -
    ↪ Force
Rename-Item C:\IA-DEV\sentiment-app-main sentiment-app
cd C:\IA-DEV\sentiment-app
Get-ChildItem -Force
```

T0 : la structure est là

Les sept dossiers `app`, `data`, `docs`, `models`, `scripts`, `tests`, `ui`, ainsi que `.github` et `.gitignore`.

5.2 Étape 2 : l'environnement (membre A, T1)

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python --version
Select-String -Path requirements.txt -Pattern "^pyarrow"
```

L'invite commence désormais par `(.venv)`. Si la dernière commande affiche une ligne `pyarrow==21.0.*`, retirez-la : elle contredit `mlflow==2.17.*`, qui exige une version plus ancienne, et pip refuserait toute l'installation (`ResolutionImpossible`). Streamlit et MLflow déclarent déjà pyarrow, pip choisira une version qui convient aux deux.

```
(Get-Content requirements.txt) | Where-Object { $_ -notmatch '^pyarrow' } | Set-Content
    ↪ requirements.txt
```

Puis l'installation, le gel des versions et le premier diagnostic :

```
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m pip freeze | Out-File -Encoding utf8 requirements.lock
python scripts/check_setup.py
python -m pytest -q
```

T1 : le point de départ

`Python 3.11.x`. L'installation dure 5 à 10 minutes : `requirements.txt` contient tout le semestre, PyTorch compris. Le diagnostic signale encore une erreur et des avertissements (pas de dépôt Git, fiches vides). pytest affiche **3 failed, 31 passed, 4 skipped** : les trois échecs réclament les fiches, les quatre tests ignorés attendent le dépôt Git.

Le script `scripts\setup.ps1` enchaîne ces étapes en une commande : `powershell -ExecutionPolicy Bypass -File scripts\setup.ps1`.

5.3 Étape 3 : les fiches de cadrage (à deux, 50 min, T2)

Dans `docs/cadrage/`, trois fiches : `cas1-pharmacie.md`, `cas2-reglement.md`, `cas3-avis-negatifs.md`. `EXEMPLE-corrige.md` montre le niveau attendu ; ne le recopiez pas. Les tests vérifient trois règles :

Règle	Test
Chaque rubrique, sauf « Approche retenue », contient au moins 20 caractères rédigés. Les lignes de consigne (qui commencent par `>`) et les cases à cocher ne comptent pas.	`test_sections_are_filled`
Une seule case cochée par fiche, écrite `- [x]` avec un x minuscule.	`test_exactly_one_approach_checked`
Le texte sous « Justification » cite au moins deux critères par leur lettre : (a) données étiquetées, (b) vérifiabilité, (c) coût par requête, (d) conséquence d'une erreur.	`test_justification_cites_two_criteria`

```
python -m pytest tests/test_cadrage.py -q
python -m pytest -q
```

T2 : les fiches sont complètes

12 passed pour `test_cadrage.py`, puis **34 passed, 4 skipped** pour la suite complète. Les deux derniers tests s'arrêtent à la première fiche fautive : une fois `cas1` corrigée, ils signalent `cas2`.

5.4 Étape 4 : le dépôt GitHub (membre A, T3)

Sur GitHub, créez un dépôt **privé** `sentiment-app`, vide : sans README, ni `.gitignore`, ni licence. Dans Settings, Collaborators, Add people, invitez le membre B, puis l'enseignant (`Larhlimiabd-ai`). Ensuite, dans le dossier du dépôt :

```
git init
git add -A
git status
git commit -m "TP1 : environnement, structure et cadrage"
git branch -M main
git remote add origin https://github.com/<compte-A>/sentiment-app.git
git push -u origin main
python -m pytest -q
```

T3 : rien de lourd ni de secret dans Git

Avant le commit, `git status` ne liste ni `.venv/`, ni `.env`, ni fichier `.joblib`. Après le push, les fichiers apparaissent sur GitHub. Le premier passage de la CI est rouge (un seul auteur) : c'est attendu. pytest affiche **37 passed, 1 failed** : il manque le commit du membre B.

L'adresse du dépôt

Remplacez `<compte-A>` par le vrai nom du compte, sans chevrons : laissés tels quels, ils provoquent une erreur 400. Le dépôt peut porter un autre nom que `sentiment-app` ; le membre B le clonera alors dans un dossier nommé `sentiment-app`. Au premier envoi, Git ouvre une fenêtre de connexion : soyez connecté à GitHub avec le compte du membre A.

5.5 Étape 5 : rejoindre le dépôt (membre B, T4)

Acceptez l'invitation, depuis le courriel de GitHub ou à l'adresse https://github.com/<compte-A>/sentiment-app/invitations. Puis, sur votre poste :

```
cd C:\IA-DEV
git clone https://github.com/<compte-A>/sentiment-app.git sentiment-app
cd sentiment-app
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/check_setup.py
```

T4 : l'environnement du membre B

`0 erreur(s)` au diagnostic. Des avertissements restent possibles tant que le binôme n'a pas fini.

Complétez ensuite votre ligne du tableau du binôme, puis commitez ce seul fichier (`requirements.lock` diffère d'un poste à l'autre, seul le membre A le commite) :

```
notepad README.md
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
```

5.6 Étape 6 : un commit par membre (membre A, T5)

```
git pull
notepad README.md
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
git shortlog -sn --all
python -m pytest tests/test_git_history.py -q
```

T5 : deux auteurs

Deux noms distincts dans `git shortlog`, puis **3 passed**.

Push refusé

`rejected (non-fast-forward)` : l'autre membre a poussé entre-temps. `git pull`, puis, en cas de conflit sur `README.md`, gardez les deux lignes, supprimez les marqueurs `<<<<<<<`, `=======` et `>>>>>>>`, puis `git add README.md`, `git commit` et `git push`.

5.7 Étape 7 : la vérification finale (à deux, T6)

```
git pull
python scripts/check_setup.py
python -m pytest -q
```

T6 : tout est vert

`0 erreur(s)`, `0 avertissement(s)`, puis **38 passed**. Sur GitHub, l'onglet Actions est vert.

5.8 Étape 8 : le test de l'enseignant, sur un clone frais (T7)

```
cd $env:TEMP
git clone https://github.com/<compte-A>/sentiment-app.git verif
cd verif
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/check_setup.py
python -m pytest -q
```

T7 : le dépôt fonctionne ailleurs que sur vos postes

`0 erreur(s)` et **38 passed**, sans aucune intervention. S'il manque quelque chose, un fichier n'a jamais été poussé.

6 TP2 : entraîner, évaluer et sauvegarder le modèle

Le TP2 ajoute un modèle de sentiment reproductible et évalué. Seule sa mesure, `models/metrics.json`, entre dans Git ; le modèle lui-même, `models/sentiment.joblib`, se réentraîne en une commande. Cinq contrôles (C1 à C5).

6.1 Installer le kit (C1)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\tp2-semaine2-kit.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\tp2-semaine2\appliquer_tp2.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp2-semaine2\appliquer_tp2.py C:\IA-DEV\sentiment-app
python -m pytest -q
```

C1 : le point de départ du TP2

La simulation liste les quatre scripts de `src/`, `tests/test_model.py`, le notebook, les deux documents du TP, puis l'ajout de `!models/metrics.json` au `.gitignore`. pytest affiche **2 failed, 38 passed, 2 skipped** : les deux échecs réclament `models/metrics.json`, les deux tests ignorés attendent le modèle.

6.2 Générer, entraîner, évaluer (C2 à C4)

```
python src/make_demo.py
python src/train.py
git status --short
python src/evaluate.py
```

C2 à C4

- C2 : `290 avis écrits`, 145 positifs et 145 négatifs, dont 100 avis mitigés et 30 neutres.
- C3 : `Vocabulaire : 316 termes`, puis `F1 macro sur le jeu de test : 0.879 (seuil 0.85)`. Dans `git status`, `models/metrics.json` apparaît ; ni `models/sentiment.joblib`, ni `data/demo_reviews.csv`. Les autres lignes sont les fichiers du kit.
- C4 : la matrice affiche `27 2` sur la ligne `negative` et `5 24` sur la ligne `positive` : 7 erreurs sur 58 avis, toutes sur des avis mitigés.

Lisez les trois erreurs affichées : un sac de mots compte les mots positifs et négatifs, mais ne voit pas le retournement d'un avis mitigé. Écrivez `docs/semaine2.md` en trois phrases : quelles erreurs, pourquoi, et ce qu'il faudrait pour les corriger.

6.3 Le vrai jeu de données, les tests, le push (C5)

```
python src/train.py --allocine
python -m pytest -q
notepad docs\semaine2.md
git add -A
git commit -m "Semaine 2 : modele de sentiment, metriques et tests"
git push
```

C5 : tout est vert

`Source : allocine, 20000 avis`, puis `F1 macro sur le jeu de test : 0.920`, et **42 passed**. L'onglet Actions passe au vert : la CI n'a jamais le modèle, ses deux tests de prédiction y sont ignorés, et les deux tests de métriques lisent `metrics.json`.

Membre B : `git pull`, `python src/make_demo.py`, `python src/train.py`, puis `python -m pytest -q` donne 42 passed. Sans modèle sur son poste, deux tests restent ignorés.

Allociné injoignable

Si le téléchargement échoue (proxy, coupure), gardez le modèle du jeu de démonstration et signalez-le dans `docs/semaine2.md`.

7 TP3 : exposer le modèle par une API

Le TP3 place le modèle derrière une API FastAPI qui appelle `predict()` du TP2 : `/health` dit si le service est prêt, `/predict` classe un avis. Sept contrôles (T14 à T20).

7.1 Installer le kit (T14)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\kit-tp3.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\kit-tp3\appliquer_tp3.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\kit-tp3\appliquer_tp3.py C:\IA-DEV\sentiment-app
pip install -r requirements.txt
python src/train.py
python -m pytest -q
```

Le script copie `api/`, `tests/test_api.py` et `docs/semaine3.md`, vérifie que `requirements.txt` contient FastAPI, uvicorn et httpx, ajoute `pythonpath = .` à `pytest.ini` pour que les tests trouvent `api/`, et complète `.github/workflows/ci.yml` : la CI installe l'API et entraîne le modèle avant les tests. `python src/train.py` remet le modèle du jeu de démonstration, celui que la CI entraîne : le TP2 s'est terminé sur Allociné.

T14 : 48 tests verts

La simulation liste les fichiers copiés, `déjà là` pour `requirements.txt`, puis les compléments de `pytest.ini` et de `ci.yml`. pytest affiche **48 passed** : les 42 du TP2, plus les six de l'API. Un avertissement `DeprecationWarning` venu de starlette est sans conséquence.

Où lancer `pip install`

Toujours dans le dossier du dépôt, jamais dans celui du kit : `requirements.txt` est dans le dépôt.

7.2 Faire répondre l'API (T15 à T18)

```
uvicorn api.main:app --reload
```

Le terminal affiche `Modèle chargé avec succès : tfidf-logreg (TP2, models/sentiment.joblib)` puis `Application startup complete`. Laissez-le ouvert et faites les essais de la section 8 dans `http://localhost:8000/docs`. Arrêtez ensuite l'API avec `Ctrl+C`.

7.3 Pousser et vérifier la CI (T19)

```
git add -A
git commit -m "Semaine 3 : API FastAPI"
git push
```

T19 : la CI entraîne puis teste

Dans l'onglet Actions, l'étape « Modèle de démonstration, pour les tests de l'API (TP3) » passe, puis l'étape « Tests » affiche 48 passed. Les avertissements `CRLF will be replaced by LF` au `git add` sont sans conséquence.

7.4 L'API sans modèle (T20), puis le compte rendu

```
Rename-Item models/sentiment.joblib sentiment.bak
uvicorn api.main:app
Rename-Item models/sentiment.bak sentiment.joblib
notepad docs\semaine3.md
git add -A
git commit -m "Semaine 3 : compte rendu"
git push
```

T20 : l'API refuse de démarrer

`FileNotFoundError: models/sentiment.joblib absent : lancez python src/train.py`, puis `Application startup failed`. Lancez la troisième commande seulement après cet échec : elle remet le modèle en place.

Les deux questions de `docs/semaine3.md` : pourquoi l'API refuse-t-elle de démarrer sans modèle plutôt que de répondre 503 ; un texte vide est refusé deux fois, par Pydantic puis par `predict()`, est-ce redondant ? Répondez avec ce que vous avez observé.

Membre B : `git pull`, `python src/make_demo.py`, `python src/train.py`, `python -m pytest -q` : 48 passed. Il fait aussi les essais de l'API sur son poste.

8 Les tests de l'API

L'API se teste de trois façons : à la main dans le navigateur, en ligne de commande, et par les tests automatiques que pytest et la CI rejouent à chaque fois. Les deux premières demandent l'API lancée (`uvicorn api.main:app`) dans un autre terminal ; les tests automatiques n'en ont pas besoin.

8.1 Dans le navigateur : la page `/docs`

FastAPI génère cette page à partir du code. Pour chaque route : la déplier, cliquer sur « Try it out », saisir le corps de la requête, puis « Execute ». La réponse s'affiche sous « Code » et « Response body », et une ligne s'ajoute dans le terminal de l'API.

Contrôle	Requête	Réponse au TP3	Réponse après le TP4
T15	`GET /health`	200, `"status": "OK"`, modèle `tfidf-logreg (TP2, ...)`	identique
T16	`POST /predict`, `{"text": "Livraison rapide, produit conforme, très satisfait"}`	200, `POSITIVE`, 0,777	identique
T17	`{"text": "Produit cassé et service injoignable, à fuir"}`	200, `NEGATIVE`, 0,7465	identique
T18	`{"text": ""}`, puis `{}`, puis `{"text": 42}`	422 dans les trois cas	identique
T22	`{"text": "  "}` (trois espaces)	422, mais `detail` est une chaîne	422, `detail` est une liste
T24	`{"text": "xqzvb wlkptr grombu"}`	200, `NEGATIVE`, 0,5162	200, `INDETERMINE`, 0,5162

8.2 En ligne de commande

`httpx`, installé avec `requirements.txt`, permet les mêmes essais depuis un terminal. Le code Python est entre guillemets doubles, les chaînes à l'intérieur entre apostrophes :

```
python -c "import httpx; print(httpx.get('http://127.0.0.1:8000/health').json())"
python -c "import httpx; r = httpx.post('http://127.0.0.1:8000/predict', json={'text': '
    ↪ Livraison rapide, produit conforme, très satisfait'}); print(r.status_code, r.
    ↪ json())"
python -c "import httpx; print([httpx.post('http://127.0.0.1:8000/predict', json=c).
    ↪ status_code for c in ({'text': ''}, {}, {'text': 42})])"
python -c "import httpx; r = httpx.post('http://127.0.0.1:8000/predict', json={'text': '
    ↪    '}); print(r.status_code, r.json())"
```

Sorties mesurées au TP3

```
{'status': 'OK', 'model': 'tfidf-logreg (TP2, models/sentiment.joblib)'}
200 {'label': 'POSITIVE', 'score': 0.777, 'text': 'Livraison rapide, produit conforme
    ↪ , très satisfait'}
[422, 422, 422]
422 {'detail': 'texte vide'}
```

Après le TP4, la dernière ligne devient `422 {'detail': [{'type': 'value_error', 'loc': ['body', 'text'], ...}]}`.

8.3 Les tests automatiques

`tests/test_api.py` contient six tests. Ils utilisent `TestClient`, qui appelle l'API en mémoire, sans serveur ; ils sont ignorés tant que le modèle n'existe pas sur le poste, et la CI entraîne le modèle avant de les lancer.

Test	Ce qu'il vérifie
`test_root`	`GET /` répond 200 et indique la page `/docs`
`test_health`	`/health` : `status` vaut `OK` et le modèle est celui du TP2
`test_predict_positif`	Un avis évident reçoit `POSITIVE`, avec un score d'au moins 0,5
`test_predict_negatif`	Un avis évident reçoit `NEGATIVE`
`test_predict_texte_vide_422`	`""` et `"  "` reçoivent 422
`test_predict_champ_manquant_422`	`{}` et `{"text": 42}` reçoivent 422

Le TP4 ajoute trois fichiers, écrits ou complétés par le binôme :

Fichier	Ce qu'il vérifie
`tests/test_limites.py`	Un test paramétré de huit cas au moins, aux frontières du contrat, avec la forme de chaque 422 ; deux propriétés hypothesis : tout texte non blanc de 1 à 2000 caractères reçoit 200, une étiquette du contrat et un score entre 0 et 1 ; tout texte de blancs reçoit 422
`tests/test_integration.py`	Le démarrage réel ; le refus de démarrer sans modèle ; 503 quand le modèle est retiré ; 500, sans le message interne, quand le modèle plante ; le refus d'un modèle aux étiquettes inconnues
`tests/test_jeu_or.py`	Les 20 avis du jeu d'or : aucune erreur franche ; une abstention est comptée, pas en échec ; une erreur connue est déclarée en `xfail(strict=True)`

```
python -m pytest -q tests/test_api.py
python -m pytest -v tests/test_api.py
python -m pytest -q --cov=api --cov-report=term-missing
```

La première commande donne `6 passed`, la deuxième affiche le nom de chaque test, la troisième la couverture de `api/`, ligne par ligne.

8.4 Lire un refus 422

FastAPI répond à une requête invalide par un 422 dont `detail` est une liste d'objets : `loc` désigne le champ fautif (`["body", "text"]`), `msg` explique le refus, `type` le classe. Au TP3, un texte fait d'espaces passe le schéma, puis `predict()` le refuse : le 422 porte alors une simple chaîne, `"texte vide"`. Deux formes pour le même refus, c'est le premier défaut que le TP4 fait trouver.

9 TP4 : tests approfondis et qualité

Le TP4 n'ajoute presque rien à l'API : il cherche ses défauts, les corrige et installe les garde-fous qui empêcheront leur retour. Le seul changement visible est un troisième verdict, `INDETERMINE`, quand le modèle n'est pas assez sûr de lui. Sept contrôles (T21 à T27). Le sujet complet est dans `TP4-ENONCE.md`, le pas à pas dans `MARCHE-A-SUIVRE-TP4.md`, tous deux copiés dans le dépôt par le kit.

9.1 Avant la séance : le jeu d'or

Vingt avis rédigés par le binôme (dix chacun), absents des données d'entraînement, chacun avec l'étiquette attendue. Au moins cinq sont difficiles : négation, ironie, avis mitigé, avis d'un autre domaine. Une erreur franche que le modèle ne sait pas éviter reste dans le jeu, avec sa raison dans la colonne `erreur_connue`.

```
text,attendu,difficulte,erreur_connue
"Commande reçue en deux jours, emballage soigné",POSITIVE,,
"Pas déçu du tout, je recommande",POSITIVE,négation,
"Arrivé cassé, bravo le transporteur",NEGATIVE,ironie,
```

9.2 Étape A : installer le kit et mesurer le départ (T21)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\tp4-tests-qualite-kit.zip" -DestinationPath C:\IA-DEV -
    ↪ Force
python C:\IA-DEV\tp4-tests-qualite\appliquer_tp4.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp4-tests-qualite\appliquer_tp4.py C:\IA-DEV\sentiment-app
pip install -r requirements-dev.txt
python src/train.py
python -m pytest -q
```

T21 : le point de départ

1 failed, 51 passed, 1 skipped. L'échec est `test_jeu_or_complet` (« jeu d'or incomplet : 0 avis sur 20 ») ; le test sauté attend ces mêmes avis (« got empty parameter set »).

Mesurez ensuite le point de départ, sans rien corriger :

```
python -m pytest -q --cov=api --cov-report=term-missing
ruff check .
mypy api
notepad docs\semaine4.md
```

Les trois mesures de départ

— Couverture : `TOTAL` à 91 %, six lignes manquantes dans `api/main.py` : 49 (le 503 quand le modèle n'est pas chargé), 77 à 79 (le 500 quand le modèle plante), 85 à 87 (le lancement direct, jamais exécuté sous pytest).

— ruff : 11 remarques. B904 deux fois dans `main.py` ; I001 et F401 dans `model.py` ; F401 et B905 deux fois dans `evaluate.py` ; I001 dans `test_api.py` ; E741 trois fois dans deux tests du TP1.

— mypy : `Success: no issues found in 4 source files`.

Ces mesures vont dans les parties 1 et 5 de `docs/semaine4.md`. Commitez : `git add -A`, puis `git commit -m "TP4 : point de depart mesure"`.

9.3 Étape B : le contrat d'entrée (T22, T23)

Complétez `tests/test_limites.py` : un test paramétré de huit cas au moins (texte vide, espaces seuls, un caractère, 2000 et 2001 caractères, champ absent, `42`, corps qui n'est pas du JSON), qui vérifie aussi la forme de chaque 422 ; puis deux propriétés hypothesis.

```
python -m pytest -q tests/test_limites.py
```

Avant la correction : deux échecs voulus

2 failed, 8 passed. Le cas des espaces reçoit bien 422, mais sous une autre forme ; la propriété des blancs échoue aussi, et hypothesis montre son plus petit contre-exemple sous « Falsifying example » : `texte='\x85'`, un caractère que `str.strip()` retire.

Corrigez ensuite le *schéma*, pas le modèle : dans `api/schemas.py`, un `field_validator` refuse un texte vide une fois ses blancs retirés, avec la même règle que `predict()` du TP2. Relancez trois fois.

T22 et T23

10 passed, trois fois de suite. Dans `/docs`, `{"text": "  "}` reçoit un 422 dont `detail` est une liste, comme `{"text": ""}`.

9.4 Étape C : l'abstention et le jeu d'or (T24, T25)

Dans `/docs`, envoyez trois textes faits de mots inventés : ils reçoivent tous `NEGATIVE` avec la même confiance, 0,5162. Sans mot connu, la représentation TF-IDF est nulle et il ne reste que l'ordonnée à l'origine de la régression logistique. Mesurez ensuite le compromis :

```
python src/courbe_seuil.py
```

La courbe du seuil

Les deux premières lignes sont :

```
seuil  répond à   part  exactitude  erreurs  vers un humain
 0.50        58   100%       0.879        7               0
```

À 0,50, le modèle répond à tout : on retrouve l'exactitude et les 7 erreurs du TP2. Sinon, le modèle n'est pas celui du jeu de démonstration : relancez `python src/train.py`.

Dans `api/model.py`, sous le seuil (constante nommée `SEUIL_ABSTENTION`, 0,60 par défaut), l'étiquette devient `INDETERMINE` et le score reste renvoyé. Copiez vos vingt avis dans `data/jeu_or.csv`, puis :

```
python src/bilan_jeu_or.py
python -m pytest -q tests/test_jeu_or.py
```

T24 et T25

Dans `/docs`, un texte inventé reçoit `INDETERMINE`. Le bilan compte les réponses justes, les abstentions et les erreurs franches déclarées ; il va dans `docs/semaine4.md`. Le test du jeu d'or n'affiche aucun `failed` : les erreurs déclarées apparaissent en `xfailed`.

9.5 Étape D : l'API telle qu'elle tourne (T26)

Écrivez `tests/test_integration.py` : démarrage réel (`with TestClient(app)`), modèle absent (chemin redirigé avec `monkeypatch`, cache vidé avec `charger_modele.cache_clear()`), modèle retiré de `app.state`, modèle en panne (`RuntimeError`, puis `ValueError("bug interne")`).

```
python -m pytest -q tests/test_integration.py
python -m pytest -q --cov=api --cov-report=term-missing --cov-fail-under=95
```

T26

Avant la correction de `main.py`, la panne `ValueError` reçoit 422 avec le message interne : c'est le défaut à trouver. Une fois la conversion de `ValueError` en 422 retirée, toute panne répond 500, et la sortie se termine par `Required test coverage of 95% reached` : 97 % sans exclusion, 100 % avec le bloc `__main__` exclu et justifié.

9.6 Étape E : qualité statique, crochet et CI (T27)

Le contrat typé. mypy ne dit rien, parce que `predict()` renvoie un `dict` : tout ce qui en sort est `Any`. Donnez un `TypedDict` au retour de `predict()` et de `SentimentClassifier.predict`, avec une étiquette `Literal["POSITIVE", "NEGATIVE", "INDETERMINE"]`. mypy signale alors `.upper()` : remplacez-le par une correspondance explicite, vérifiée au démarrage.

ruff. `ruff check . --fix` règle l'essentiel. Le reste se corrige à la main : `strict=True` dans les deux `zip()` de `evaluate.py`, `from None` sur le `raise` du 500, et le renommage de `l` en `ligne` dans deux tests du TP1, sans remplacement automatique, qui abîmerait les « l' » des messages :

Fichier, ligne	Correction
`test_cadrage.py`, 51	`l for l in body.splitlines() if l.strip() ...` devient `ligne for ligne in body.splitlines() if ligne.strip() ...`, et de même pour les deux `startswith`
`test_environment.py`, 38	`[l.strip() for l in` devient `[ligne.strip() for ligne in`
`test_environment.py`, 39	`[l for l in lines if l and not l.startswith("#")]` devient `[ligne for ligne in lines if ligne and not ligne.startswith("#")]`

Le crochet. Installez-le, puis vérifiez qu'il refuse un commit fautif :

```
pre-commit install
Add-Content src\courbe_seuil.py "import os"
git add src\courbe_seuil.py
git commit -m "essai"
git restore --staged src\courbe_seuil.py
git restore src\courbe_seuil.py
```

La ligne `ruff` passe en `Failed` et le commit n'est pas créé ; les deux dernières commandes annulent l'essai.

La CI. Dans `.github\workflows\ci.yml`, remplacez les deux lignes de l'étape « Tests » par les quatre étapes de `ci-tp4.yml`, avec le même retrait de six espaces. Chacune active l'environnement virtuel, comme les étapes du TP1 :

```
      - name: Outils de qualité (TP4)
        run: . .venv/bin/activate && pip install -r requirements-dev.txt
      - name: ruff
        run: . .venv/bin/activate && ruff check .
      - name: mypy
        run: . .venv/bin/activate && mypy api
      - name: Tests et couverture
        run: . .venv/bin/activate && pytest -q --cov=api --cov-report=term-missing --cov-
    ↪ fail-under=95
```

```
ruff check .
mypy api
python -m pytest -q
git add -A
git commit -m "Semaine 4 : tests approfondis et qualite"
git push
```

T27 : tout est vert, en local et en CI

`All checks passed!`, puis `Success: no issues found in 4 source files`, puis aucun `failed` : 83 passed et 2 xfailed avec le corrigé, vos nombres pouvant être plus grands. Dans l'onglet Actions, les étapes ruff, mypy et « Tests et couverture » passent.

Membre B : `git pull`, `pip install -r requirements-dev.txt`, `python src/train.py`, `pre-commit install` (le crochet s'installe dans chaque clone), puis `python -m pytest -q`.

10 TP5 : l'interface Streamlit

Le TP5 donne au service client une page web pour analyser un avis, ou tout un fichier d'avis. Sept contrôles (T28 à T34).

10.1 Le rôle de l'interface

La page est un client de l'API, comme n'importe quel programme : elle envoie `POST /predict` et traduit chaque réponse en message clair, y compris l'abstention et les erreurs. Elle n'importe jamais `predict()` ni le fichier `.joblib` (troisième règle d'architecture) : un modèle réentraîné ne demande aucune modification de l'interface. Deux programmes tournent donc en même temps : l'API sur le port 8000, la page sur le port 8501.

Une application Streamlit est un script Python ordinaire, réexécuté de haut en bas à chaque

action de l'utilisateur. Les composants (`st.text_area`, `st.button`, `st.file_uploader`) renvoient leur valeur courante ; les affichages (`st.metric`, `st.warning`, `st.error`) écrivent dans la page.

10.2 Installer le kit (T28)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\tp5-interface-kit.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\tp5-interface\appliquer_tp5.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp5-interface\appliquer_tp5.py C:\IA-DEV\sentiment-app
python src/train.py
python -m pytest -q
```

T28 : douze tests de plus, aucun échec

La simulation liste `ui/`, `tests/test_client.py`, `tests/test_interface.py`, `data/avis_exemple.csv`, `docs/semaine5.md`, puis l'ajout de `!data/avis_exemple.csv` au `.gitignore` et de Streamlit à la CI. pytest n'affiche aucun `failed` : douze tests de plus qu'à la fin du TP4, soit 95 passed et 2 xfailed avec le corrigé du TP4. Streamlit est déjà installé par `requirements.txt`.

10.3 La structure de l'interface (étape B, 20 min)

L'interface tient en trois fichiers, et chacun a un seul rôle : la page affiche, le client parle à l'API, l'API décide. Comprendre cette séparation, c'est comprendre pourquoi un modèle réentraîné, ou une API déplacée sur un autre serveur, ne demande aucune modification de la page.

10.3.1 Trois fichiers, trois rôles

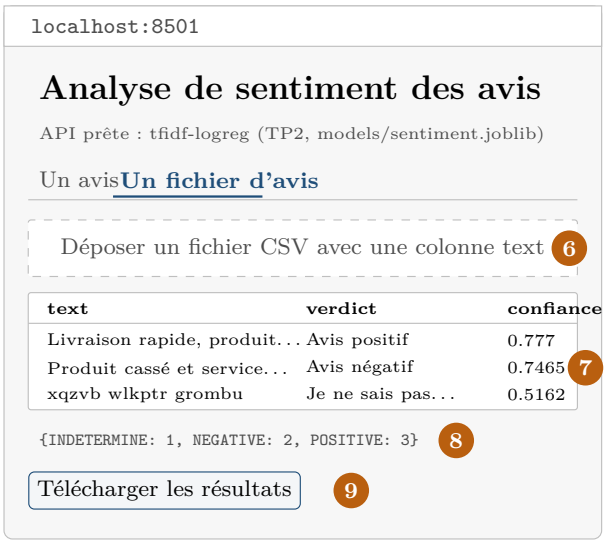
Fichier	Rôle
`ui/__init__.py`	Fait de `ui/` un paquet Python : `from ui import client` devient possible
`ui/client.py`	Parle à l'API : santé, prédiction, lot d'avis, résumé ; traduit chaque réponse
`ui/app.py`	La page Streamlit : elle affiche, elle ne décide rien
`tests/test_client.py`	Sept tests du client, contre l'API en mémoire
`tests/test_interface.py`	Cinq tests de la page, avec l'outil de test de Streamlit (`AppTest`)
`data/avis_exemple.csv`	Six avis pour essayer l'onglet « Un fichier d'avis »

La page n'importe jamais `predict()`, ni `api/`, ni le fichier `.joblib` : elle ne connaît que `ui.client`. Le client, lui, ne connaît que l'adresse de l'API et ses codes de réponse.

10.3.2 La page, zone par zone

La page a deux onglets. À gauche, l'onglet « Un avis » après une analyse ; à droite, l'onglet « Un fichier d'avis » après le chargement de `data/avis_exemple.csv`.

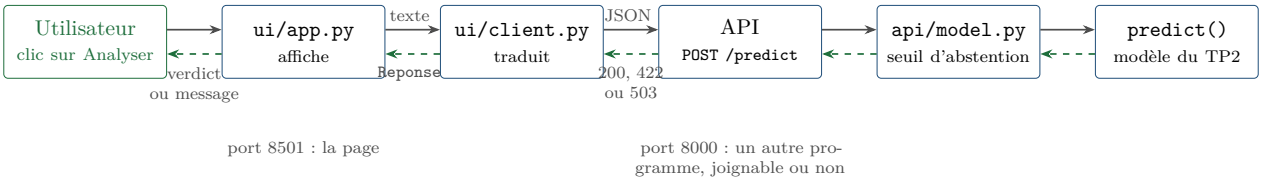




Zone	Ce que voit l'utilisateur	Appel Streamlit	Fonction du client
1	Le titre de la page	`st.title`	
2	L'état de l'API, vérifié à chaque réexécution	`st.caption`, ou `st.error`	`sante()`
3	Les deux onglets	`st.tabs`	
4	Le champ de l'avis et le bouton	`st.text_area`, `st.button`	
5	Le verdict et la confiance ; l'abstention, le refus ou l'erreur	`st.metric`, `st.info`, `st.warning`, `st.error`	`predire()`
6	Le dépôt du fichier CSV	`st.file_uploader`	
7	Le tableau des verdicts	`st.dataframe`	`predire_lot()`
8	Le résumé par verdict	`st.write`	`resumer()`
9	Le téléchargement des résultats	`st.download_button`	

10.3.3 Le chemin d'une requête

Un clic sur « Analyser » traverse toute l'application, puis la réponse revient en sens inverse, traduite à chaque étage :



La page et l'API sont deux programmes distincts. Entre les deux, tout peut arriver : l'API arrêtée, le modèle absent, un texte refusé. C'est le client qui transforme chacun de ces cas en message, pour que la page n'ait jamais à interpréter un code HTTP.

10.3.4 Le code de la page, bloc par bloc

`ui/app.py` se lit de haut en bas, en quatre blocs.

Bloc 1, les imports. Streamlit ajoute au chemin de Python le dossier du script, `ui/`, et non la racine du dépôt : sans la ligne `sys.path.insert`, `from ui import client` échouerait.

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))  # pour importer ui.client

import pandas as pd
import streamlit as st

from ui import client
```

Bloc 2, la page et l'état de l'API. La connexion est créée par `client.creer_session()`, et non directement : c'est ce point que les tests remplacent. `sante()` interroge `/health` ; la réponse s'affiche en haut de la page, en légende si l'API est prête, en erreur sinon.

```
st.set_page_config(page_title="Analyse de sentiment")
st.title("Analyse de sentiment des avis")

session = client.creer_session()
etat = client.sante(session)
if etat.ok:
    st.caption(f"API prête : {etat.message}")
else:
    st.error(etat.message)
```

Bloc 3, l'onglet « Un avis ». Le bouton déclenche `predire()`, qui renvoie une `Reponse`. Selon son contenu, la page choisit l'affichage : le verdict et son pourcentage, un encadré de plus pour l'abstention, un avertissement pour un texte refusé, une erreur pour tout le reste.

```
onglet_avis, onglet_lot = st.tabs(["Un avis", "Un fichier d'avis"])

with onglet_avis:
    texte = st.text_area("Avis client")
    if st.button("Analyser"):
        reponse = client.predire(session, texte)
        if reponse.ok and reponse.label is not None and reponse.score is not None:
            st.metric(client.VERDICTS[reponse.label], f"{reponse.score:.0%}")
            if reponse.label == "INDETERMINE":
                st.info("La confiance est sous le seuil d'abstention : un humain doit trancher.")
        elif reponse.message == client.REFUS:
            st.warning(reponse.message)
        else:
            st.error(reponse.message)
```

Bloc 4, l'onglet « Un fichier d'avis ». Le fichier est lu par pandas ; sans colonne `text`, la page le dit et s'arrête là. Sinon, `predire_lot()` envoie chaque avis à l'API, deux colonnes s'ajoutent au tableau, et `resumer()` compte les verdicts. Le bouton de téléchargement renvoie le tableau complété, au format CSV.

```
with onglet_lot:
    fichier = st.file_uploader("Fichier CSV avec une colonne text", type="csv")
    if fichier is not None:
        avis = pd.read_csv(fichier)
        if "text" not in avis.columns:
            st.error("Colonne text absente du fichier.")
        else:
            reponses = client.predire_lot(session, avis["text"].astype(str).tolist())
            avis["verdict"] = [client.VERDICTS[r.label] if r.ok and r.label else r.message for r
    ↪ in reponses]
            avis["confiance"] = [r.score for r in reponses]
            st.dataframe(avis, use_container_width=True)
            st.write(client.resumer(reponses))
            st.download_button("Télécharger les résultats", avis.to_csv(index=False),
                               file_name="resultats.csv", mime="text/csv")
```

10.3.5 Le client : un contrat simple pour la page

Toute réponse de l'API devient une `Reponse` : soit un verdict (`ok` vrai, une étiquette et un score), soit un message prêt à afficher (`ok` faux).

```
@dataclass
class Reponse:
    """Ce que l'interface affiche : un verdict, ou un message d'erreur lisible."""

    ok: bool
    label: str | None = None
    score: float | None = None
    message: str = ""


def creer_session() -> httpx.Client:
    """La connexion à l'API. Les tests la remplacent par un client en mémoire."""
    return httpx.Client(base_url=API_URL, timeout=10)
```

Réponse de l'API	`Reponse` renvoyée	**Affichage de la page**
200, `{label, score}`	`ok=True`, l'étiquette et le score	Le verdict et son pourcentage ; un encadré de plus pour `INDETERMINE`
422	`ok=False`, `message=REFUS`	Avertissement « Texte refusé »
503	`ok=False`, `message=SANS_MODELE`	Erreur « Modèle non chargé »
Connexion impossible	`ok=False`, `message=INJOIGNABLE`	Erreur « API injoignable »
Tout autre code	`ok=False`, « Erreur interne de l'API »	Erreur, avec le code

10.3.6 Le cycle de Streamlit

À chaque action de l'utilisateur, un clic, un fichier déposé, Streamlit réexécute tout le script, du haut vers le bas. Trois conséquences :

— La connexion est recréée et la santé de l'API revérifiée à chaque fois : c'est pourquoi l'arrêt de l'API se voit dès le rechargement de la page, avant même d'analyser un avis.

— Les composants gardent leur valeur d'une exécution à l'autre : le texte saisi et le fichier déposé ne sont pas perdus.

— Le lot est recalculé à chaque réexécution, y compris au clic sur « Télécharger ». Sans conséquence pour six avis, coûteux pour dix mille : mettre ce calcul en cache avec `@st.cache_data` est une piste d'amélioration.

10.3.7 Comment les tests se branchent

`tests/test_client.py` passe au client un `TestClient`, qui fait tourner l'API en mémoire. `tests/test_interface.py` va plus loin : il remplace `client.creer_session` par ce même `TestClient`, puis exécute la page avec `AppTest`, saisit un texte, clique sur le bouton, et lit ce que la page affiche (`page.metric`, `page.warning`, `page.error`). Ni l'API ni la page n'ont besoin d'être lancées : pytest et la CI vérifient tout, à chaque push.

10.4 Lancer et analyser (T29 à T31)

Deux terminaux, dans le dépôt, environnement activé :

```
uvicorn api.main:app --reload
```

```
streamlit run ui/app.py
```

Au tout premier lancement, Streamlit demande une adresse électronique dans le terminal : appuyez sur Entrée, ou lancez avec `--server.headless true`. La page s'ouvre sur `http://localhost:8501`.

T29 à T31

Contrôle	Ce qu'on fait	Ce que la page affiche
T29	On ouvre la page	`API prête : tfidf-logreg (TP2, models/sentiment.joblib)`
T30	« Livraison rapide, produit conforme, très satisfait »	Avis positif, 78 %
T30	« Produit cassé et service injoignable, à fuir »	Avis négatif, 75 %
T31	« xqzvb wlkptr grombu »	Je ne sais pas : à faire vérifier par un humain, 52 %, et le message « la confiance est sous le seuil d'abstention »

10.5 Provoquer les erreurs (T32)

Saisissez trois espaces, puis « Analyser ». Arrêtez ensuite l'API (`Ctrl+C` dans son terminal) et rechargez la page. Relancez l'API pour la suite.

T32 : chaque erreur a son message

Les espaces donnent l'avertissement « Texte refusé : vide, fait d'espaces, ou de plus de 2000 caractères » (réponse 422). L'API arrêtée donne, en haut de la page, « API injoignable : lancez d'abord uvicorn api.main :app ».

10.6 Un fichier d'avis (T33)

Dans l'onglet « Un fichier d'avis », chargez `data/avis_exemple.csv`.

T33 : le lot

Un tableau de six avis avec leur verdict et leur confiance, le résumé `{'INDETERMINE': 1, 'NEGATIVE': 2, 'POSITIVE': 3}`, et le bouton « Télécharger les résultats », qui produit `resultats.csv`. Un avis sur six part vers un humain.

10.7 Compte rendu, push, CI (T34)

Les trois questions de `docs/semaine5.md` : pourquoi l'interface n'importe-t-elle jamais le modèle ? Que montre-t-elle quand le modèle hésite, et pourquoi est-ce mieux qu'un verdict forcé ? Que voit l'utilisateur quand l'API tombe ?

```
git add -A
git commit -m "Semaine 5 : interface Streamlit"
git push
```

T34 : la CI est verte

La CI installe Streamlit et rejoue les douze tests du TP5. Pendant l'installation des outils du TP4, pip peut écrire `streamlit 1.39.1 requires packaging<25` : c'est sans conséquence, les tests de l'interface passent.

Membre B : `git pull`, `python src/train.py`, `python -m pytest -q`, puis les essais T29 à T33 sur son poste.

11 Vérification et notation par l'enseignant

11.1 L'accès aux dépôts

Chaque dépôt est privé. Le membre A y invite le compte `Larhlimiabd-ai` (Settings, Collaborators, Add people). L'enseignant accepte l'invitation depuis le courriel de GitHub ou depuis `https://github.com/<compte-A>/<depot>/invitations` ; elle expire au bout de sept jours. Sur le poste de l'enseignant, l'adresse des dépôts porte son compte, pour que Git n'envoie pas d'autres identifiants : `https://Larhlimiabd-ai@github.com/<compte-A>/<depot>.git`.

11.2 En continu : la CI

La CI de chaque dépôt teste chaque push sur une machine neuve. L'onglet Actions montre, sans rien installer, où en est chaque binôme ; Insights, puis Contributors, montre les deux membres.

11.3 À la date de remise : la relève automatique

`releve_binomes.py` vérifie tous les dépôts d'un coup. `binomes.csv` contient deux colonnes, `binome` et `depot`, une ligne par binôme. Le script se lance dans l'environnement du cours (Python 3.11, avec `requirements.txt` et `requirements-dev.txt` installés) :

```
python releve_binomes.py binomes.csv --tp 3 --tests
python releve_binomes.py binomes.csv --tp 4 --tests --date-limite "2026-10-25 23:59"
python releve_binomes.py binomes.csv --tp 5 --tests
```

Pour chaque binôme, il reclone le dépôt à neuf dans `releve\`, se place sur le dernier commit avant la date limite et compte à part les commits plus tardifs. Il écrit `releve/releve-tpN.md`, à lire, et `releve/releve-tpN.csv`, pour un tableur.

Colonne	Ce qu'elle dit
`auteurs`	Le nombre d'auteurs et leurs noms
`fichiers interdits`	`.venv`, modèle, `data/demo_reviews.csv` ou `.env` versionnés
`livrables`	Ce qui manque : fiches, `metrics.json`, comptes rendus encore à l'état de canevas, jeu d'or incomplet, pre-commit
`tests du kit retouchés`	Les tests fournis modifiés depuis leur ajout
`F1`, `pytest`	Avec `-tests` : l'entraînement et les tests rejoués
`couverture`, `ruff`, `mypy`	Au TP4, avec `-tests`

Lire la colonne « tests du kit retouchés »

Un test vert ne prouve rien si le binôme a modifié le test : l'essai du script sur un dépôt fautif l'a montré, avec un seuil de F1 abaissé dans `test_model.py`. Regardez chaque fichier signalé avec `git log -p -- tests/<fichier>` dans le clone de `releve\`. Au TP4, trois retouches sont légitimes : l'ordre des imports de `test_api.py` remis par ruff, ses étiquettes mises à jour après l'abstention, et le renommage de `l` en `ligne` dans `test_cadrage.py` et `test_environment.py`. Avec `-tests`, le script exécute le code des étudiants : lancez-le dans l'environnement du cours.

11.4 Le clone frais, à la main

Pour un binôme, ou en cas de doute sur la relève :

```
cd $env:TEMP
git clone https://Larhlimiabd-ai@github.com/<compte-A>/<depot>.git verif
cd verif
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/check_setup.py
python src/make_demo.py
python src/train.py
python -m pytest -q
git shortlog -sn --all
git log --format="%h %ad %an : %s" --date=short
git log --all --oneline -- models/sentiment.joblib .venv
```

Ce que l'enseignant doit voir

`0 erreur(s)` ; F1 macro 0,879 ; le compte de tests du TP rendu (48 passed au TP3) ; deux noms dans `shortlog` ; un historique où chaque membre a commité dans chaque TP ; rien pour la dernière commande, qui prouve que le modèle et l'environnement n'ont jamais été commités. Ensuite, `uvicorn api.main:app` et les essais T15 à T20 dans `/docs`. Au TP4, ajoutez `pip install -r requirements-dev.txt`, `ruff check .`, `mypy api` et `python -m pytest -q --cov=api --cov-fail-under=95`. Au TP5, `streamlit run ui/app.py` et les essais T29 à T33. Gardez l'environnement de ce clone : il contient tout le semestre, PyTorch compris, et sert à la relève des autres binômes.

11.5 Les grilles de notation

Chaque TP est noté sur 10, s'il est tiré au sort. Les grilles sont celles des énoncés.

TP	Critère	Pts	Vérification
TP1	Fiches justifiées par deux critères, rubriques remplies	4	Lecture, `test_cadrage.py`
	Dépôt clonable, installation sans erreur	2	Clone frais
	Structure et `.gitignore` conformes, `.venv/` absent de l'historique	2	`test_environment.py`, `git log`
	Un commit par membre, messages explicites	2	`git shortlog`, `git log`
TP2	Modèle entraîné, sauvegardé, rechargeable par `predict()`	3	Clone frais, `src/predict.py`
	Évaluation correcte : découpage avant vectorisation, mesures sur le jeu de test	3	Code, `metrics.json`
	Les 4 tests du modèle, CI verte	2	pytest, Actions
	Analyse des erreurs	2	`docs/semaine2.md`
TP3	L'API répond : `/health`, avis positif, avis négatif	3	T15 à T17
	Codes d'erreur justes : 422, refus de démarrer sans modèle	2	T18, T20
	Tests verts en local et en CI	2	T14, T19
	Les deux réponses, appuyées sur des observations	3	`docs/semaine3.md`
TP4	Contrat d'entrée : cas limites, propriétés, correction dans le schéma	2	T22, T23
	Abstention : seuil mesuré et justifié, jeu d'or honnête	2,5	T24, T25
	Intégration : démarrage, modèle absent, 503, 500 ; couverture de 95 % sans exclusion abusive	2	T26
	Qualité statique et automatisation : contrat typé, ruff et mypy muets, pre-commit, CI verte	2	T27
	`docs/semaine4.md` : chaque défaut relié au test qui l'a trouvé, chiffres du binôme	1,5	T21
TP5	La page fonctionne : un avis, verdict, confiance, abstention	3	T29 à T31
	Les erreurs traduites pour l'utilisateur : 422, 503, API injoignable	2	T32
	Le lot d'avis : tableau, résumé, téléchargement	2	T33
	Tests du client et de la page verts, en local et en CI	1,5	T28, T34
	`docs/semaine5.md` : les trois réponses, appuyées sur des observations	1,5	Lecture

11.6 Les signaux d'alerte

— Un seul commit géant qui contient tout un TP, ou un membre absent de l'historique d'un TP.

— Des fiches ou des comptes rendus identiques à ceux d'un autre binôme.

— Les valeurs du corrigé recopiées à la place des mesures du binôme.

— Un test fourni par un kit modifié pour passer, des `# noqa` ou des `# pragma: no cover` en série, un jeu d'or sans cas difficile.

12 Les messages d'erreur rencontrés

Tous ces messages sont apparus pendant la mise au point de cette démarche, sur de vrais postes. Ceux marqués *attendu* sont des étapes normales du parcours : ils désignent le travail qui reste à faire.

Message ou symptôme	Cause	Correction
Python et environnement		
`'pytest' n'est pas reconnu`, `No module named pytest`	Environnement virtuel non activé	`.venv\Scripts\Activate.ps1`
`Activate.ps1` : l'exécution de scripts est désactivée	Politique d'exécution de Windows	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, une fois
`'deactivate' n'est pas reconnu`	Invite cmd, aucun environnement actif	Sans conséquence. Dans cmd : `rmdir /s /q .venv`, `.venv\Scripts\activate.bat`
`Compiler cl cannot compile programs`, `metadata-generation-failed` (pandas)	Python plus récent que 3.11 : pas de version précompilée, pip tente de compiler	Recréer l'environnement : `py -3.11 -m venv .venv`
`ResolutionImpossible` : `pyarrow==21.0.*` contre `mlflow 2.17`	Ligne `pyarrow` contradictoire dans le `requirements.txt` du kit TP1	Retirer la ligne (TP1, étape 2), et la corriger dans le dépôt du cours
`No module named 'joblib'`	Dépendances jamais installées dans cet environnement (clone de vérification)	`pip install -r requirements.txt`
`Could not open requirements file: requirements-dev.txt`	Kit du TP4 pas encore installé	`appliquer_tp4.py`, puis `pip` dans le dépôt
`Could not open requirements file: requirements.txt`	`pip` lancé dans le dossier du kit	Se placer dans le dossier du dépôt
`DeprecationWarning` (starlette, anyio, joblib)	Versions des bibliothèques	Sans conséquence
Git et GitHub		
`1 auteur(s) de commits, 2 attendus`	Le membre B n'a pas encore commité, ou les deux postes ont le même `user.name`	*Attendu* après le premier push. Commit de B sous son identité
`remote origin already exists`	Adresse déjà enregistrée	`git remote set-url origin <adresse>`
`Permission ... denied to` un autre compte (403)	Windows envoie les identifiants d'un autre compte	Compte dans l'adresse : `https://<compte>@github.com/...` ; ou `cmdkey /delete:git:https://github.com`
`src refspec main does not match any`	Aucun commit, ou branche `master`	`git commit`, puis `git branch -M main`
`Updates were rejected`	Dépôt GitHub créé avec un README	Le recréer vide, ou `git pull origin main --allow-unrelated-histories`
`The requested URL returned error: 400`	Chevrons `<depot>` laissés dans l'adresse	La vraie adresse du dépôt
`Repository not found`	Invitation non acceptée, ou mauvais compte	Accepter l'invitation ; compte dans l'adresse
`CRLF will be replaced by LF`	Fins de ligne de Windows	Sans conséquence
Chemins et archives		
`ne ressemble pas au dépôt du TP1 (README.md et tests/ attendus)`	Chemin relatif faux : l'Explorateur a ajouté un dossier	Donner le chemin complet du dépôt
`Expand-Archive` : le chemin d'accès n'existe pas	Archive ailleurs, ou renommée par le navigateur	`Get-ChildItem $HOME -Recurse -Filter ...`

Message ou symptôme	Cause	Correction
`Rename-Item` : l'élément n'existe pas	Rien à renommer : modèle jamais entraîné, ou mise de côté non faite	`Get-ChildItem models`, puis `python src/train.py`
`unrecognized arguments: puis git status --short`	Deux commandes d'une case de tableau copiées comme une seule	Une commande par ligne
`45 passed` au lieu de 48	Dépôt d'une autre chaîne de kits, où le TP3 était l'interface	Utiliser les kits de ce guide
Tests et contrôles		
`rubriques vides ou trop courtes`, `cocher exactement une approche`, `la justification doit citer au moins deux critères`	*Attendu* : fiches pas encore remplies (T1)	TP1, étape 3
`jeu d'or incomplet : 0 avis sur 20`, `got empty parameter set`	*Attendu* : jeu d'or vide (T21)	Les vingt avis dans `data/jeu_or.csv`
ruff : B904, I001, F401, B905, E741	*Attendu* : point de départ du TP4	TP4, étape E
`2 failed` et `Falsifying example: texte='\x85'`	*Attendu* : le défaut que l'étape B fait trouver	`field_validator` dans le schéma
API et CI		
`FileNotFoundError: models/sentiment.joblib absent` au démarrage	Pas de modèle : contrôle T20 s'il a été mis de côté, sinon jamais entraîné	Remettre le modèle, ou `python src/train.py`
`[WinError 10048]` : adresse déjà utilisée	Une autre API tourne déjà	Fermer l'autre terminal, ou `--port 8001`
CI : `No module named 'api'`	`pytest.ini` sans `pythonpath = .` : la commande `pytest` seule ne trouve pas `api/`	Le kit du TP3 ajoute la ligne
CI verte, mais tests de l'API ignorés	La CI n'entraîne pas le modèle	L'étape d'entraînement avant les tests
CI du TP4 : outils introuvables	Étapes sans activation de l'environnement virtuel	Les quatre étapes de `ci-tp4.yml`
Interface Streamlit		
Streamlit demande une adresse électronique	Accueil du premier lancement	Entrée, ou `--server.headless true`
« API injoignable » en haut de la page	L'API n'est pas lancée	`uvicorn api.main:app` dans un autre terminal
Port 8501 déjà utilisé	Une autre page Streamlit tourne	La fermer, ou `--server.port 8502`
`streamlit 1.39.1 requires packaging<25`	Les outils du TP4 installent un `packaging` plus récent	Sans conséquence : les tests de l'interface passent

A Récapitulatif des contrôles

Contrôle	Commande ou essai	Résultat attendu
TP1		
T0	`Get-ChildItem -Force`	Sept dossiers, `.github`, `.gitignore`

Contrôle	Commande ou essai	Résultat attendu
T1	`python -m pytest -q`	3 failed, 31 passed, 4 skipped
T2	`python -m pytest tests/test_cadrage.py -q`	12 passed
T3	`git status` avant le commit	Ni `.venv/`, ni `.env`, ni `.joblib`
T4	`python scripts/check_setup.py` (membre B)	0 erreur(s)
T5	`git shortlog -sn --all`, puis `python -m pytest tests/test_git_history.py -q`	Deux noms, puis 3 passed
T6	`check_setup.py`, puis `python -m pytest -q`	0 erreur(s), 0 avertissement(s) ; 38 passed ; CI verte
T7	La suite complète sur un clone frais	38 passed, sans intervention
TP2		
C1	`python -m pytest -q`	2 failed, 38 passed, 2 skipped
C2	`python src/make_demo.py`	290 avis, 145 de chaque classe
C3	`python src/train.py`, puis `git status --short`	F1 macro 0,879 ; `metrics.json` seul, sans le modèle
C4	`python src/evaluate.py`	7 erreurs sur 58, toutes sur des avis mitigés
C5	`python src/train.py --allocine`, puis `python -m pytest -q`	F1 macro 0,920 ; 42 passed ; CI verte
TP3		
T14	`python -m pytest -q`	48 passed
T15	`GET /health`	200, `status` OK
T16	`POST /predict`, avis positif	200, `POSITIVE`, 0,777
T17	`POST /predict`, avis négatif	200, `NEGATIVE`, 0,7465
T18	Texte vide, champ absent, nombre	422 dans les trois cas
T19	`git push`, onglet Actions	Entraînement, puis 48 passed
T20	`uvicorn api.main:app` sans modèle	`FileNotFoundError`, l'API ne démarre pas
TP4		
T21	`python -m pytest -q`	1 failed, 51 passed, 1 skipped
T22	`/docs`, trois espaces	422, `detail` en liste
T23	`python -m pytest -q tests/test_limites.py`, trois fois	10 passed
T24	`/docs`, mots inventés	`INDETERMINE`
T25	`python -m pytest -q tests/test_jeu_or.py`	Aucun failed ; erreurs connues en xfailed
T26	`python -m pytest -q --cov=api --cov-fail-under=95`	Pannes en 500 ; seuil de 95 % atteint
T27	`ruff check .`, `mypy api`, `python -m pytest -q`, push	Muets ; aucun failed ; CI verte
TP5		
T28	`python -m pytest -q`	Aucun failed, douze tests de plus
T29	`streamlit run ui/app.py`	« API prête »
T30	Un avis positif, un avis négatif	Avis positif 78 %, avis négatif 75 %
T31	Des mots inventés	« Je ne sais pas », 52 %
T32	Trois espaces ; l'API arrêtée	« Texte refusé » ; « API injoignable »
T33	`data/avis_exemple.csv`	3 positifs, 2 négatifs, 1 à vérifier ; téléchargement
T34	`git push`	CI verte

B PowerShell, cmd et bash

Action	PowerShell	cmd	bash
Créer l'environnement	`py -3.11 -m venv .venv`	`py -3.11 -m venv .venv`	`python3.11 -m venv .venv`
L'activer	`.venv\Scripts\Activate.ps1`	`.venv\Scripts\activate.bat`	`source .venv/bin/activate`
Le supprimer	`Remove-Item -Recurse -Force .venv`	`rmdir /s /q .venv`	`rm -rf .venv`
Renommer un fichier	`Rename-Item a b`	`ren a b`	`mv a b`
Lister, fichiers cachés compris	`Get-ChildItem -Force`	`dir /a`	`ls -a`
Afficher un fichier	`Get-Content f`	`type f`	`cat f`
Dossier temporaire	`cd $env:TEMP`	`cd %TEMP%`	`cd /tmp`
Variable d'environnement	`$env:API_URL = "..."`	`set API_URL=...`	`export API_URL=...`
Modifier un fichier	`notepad f`	`notepad f`	`nano f`

Les commandes `python`, `pip`, `git`, `uvicorn`, `ruff`, `mypy` et `pre-commit` s'écrivent de la même façon partout ; seuls les séparateurs de chemin changent (`\` sous Windows, `/` ailleurs, `/` étant accepté par Python sous Windows).